Jev-Backed Function Calling: Agentic In-Context Learning with Cumulative Decision Feedback

Minhyeong Yu
Enhans
minhyeong@enhans.ai

Abstract

Function-calling models must choose a function and input values that match the user's request. Correct format alone does not ensure correct choices. We present Jev-Backed Function Calling (**JBFC**), an inference-time method that uses TypeSafe's System One (Decision) model *Jev* to check an LLM's proposed call. *Jev* checks function choice and, when a call is present, its arguments with binary questions. If a required check fails, we add the proposal and judgments to a growing history placed before the original user request. The LLM uses this feedback to revise both the function and arguments. Acceptance requires all applicable binary checks and the schema check to pass on the same proposal. The repair limit is fixed at three. Our code compares seven methods on a 3,051-example Berkeley Function Calling Leaderboard (BFCL) subset, measuring call correctness, function choice, argument correctness, time, and cost. In one run with `gpt-5-nano`, **JBFC** raises call accuracy over direct calling from 63.8% to 74.6%, mostly by avoiding unnecessary calls, at about four times the cost. It has the highest call accuracy of the seven methods, and keeping all earlier feedback adds 1.1 points over showing only the latest rejection. With the larger `gpt-5.4-mini`, it raises mean accuracy over three runs from 81.1% to 82.1%.

1 Introduction

To carry out a user's request correctly, a function-calling system must choose the right function and provide the right argument values. A language model, called the generator, makes these choices. However, a call can have the required format and still use the wrong function or values. For example, a travel assistant may choose the wrong booking function or swap the departure and destination. A JSON schema check verifies required inputs and their types, but cannot guarantee that the call matches the user's request.

Tool-use training and evaluation have improved function-calling systems [2, 3, 1]. Even so, the generated function and arguments can still be wrong. We study whether checks from a separate decision model can guide the LLM to improve its next proposal before execution.

JBFC uses *Jev*, TypeSafe's System One model, to answer binary questions about an LLM's function choice and arguments [12]. When a check fails, we keep the proposed call and its judgments and place all earlier records before the original user request. The LLM then proposes a new call using this accumulated feedback. It can change both the function and argument values. **JBFC** accepts a proposal only when all applicable binary checks and the schema check pass on that proposal.

Earlier work already uses checks and feedback to fix calls. ToolVerifier checks tool choice and parameters [9]; ToolCritic gives feedback on tool-use errors [10]. ReAct uses reasoning, actions, and observations to guide later steps [16]. **JBFC** also carries feedback forward in the prompt, but its feedback comes from checking proposed calls before execution. Our contribution is this specific use of two *Jev* judgments and accumulated prompts, together with an evaluation of its effects and costs.

We compare **JBFC** with direct calling, schema repair, self-revision, single-check versions, and a latest-feedback version. A fixed repair limit of three controls the comparison. We measure errors fixed or introduced by repair, and keep call correctness separate from acceptance, time, and cost.

2 Related Work

Tool use and benchmarks. Gorilla studies API-call generation [2]; ToolLLM studies tool-use training and evaluation [3]. BFCL tests function calling across many settings [1]. We use its single-turn Python categories to study decisions made before a function runs. API-Bank offers dialogues with executable API calls [4]. StableToolBench uses caching and simulation to handle changing APIs [5]. These suggest broader tests beyond our BFCL subset.

Checks and feedback. Self-Refine uses self-generated feedback [6], Reflexion uses written feedback [7], and CRITIC uses feedback from tools [8]. Self-correction can turn correct answers into wrong ones [11], so we measure both fixes and new errors. ToolVerifier compares alternatives to check tool choice and parameters separately [9]. ToolCritic identifies tool-use errors and returns feedback [10]. **JBFC** checks complete proposals with a fixed external model through an API and limits repairs.

Accumulated prompts. ReAct combines reasoning with actions and observations from an environment [16]. Our method uses a similar idea of carrying earlier feedback into the next prompt. Each record contains a proposed function call and verification results, rather than an execution result. The LLM receives all records before the unchanged user request. We do not require a generated reasoning trace or run functions during repair. Many-shot in-context learning often improves as prompts grow to hundreds or thousands of examples, and model-generated rationales kept only when their answers are correct can stand in for human-written ones [17].

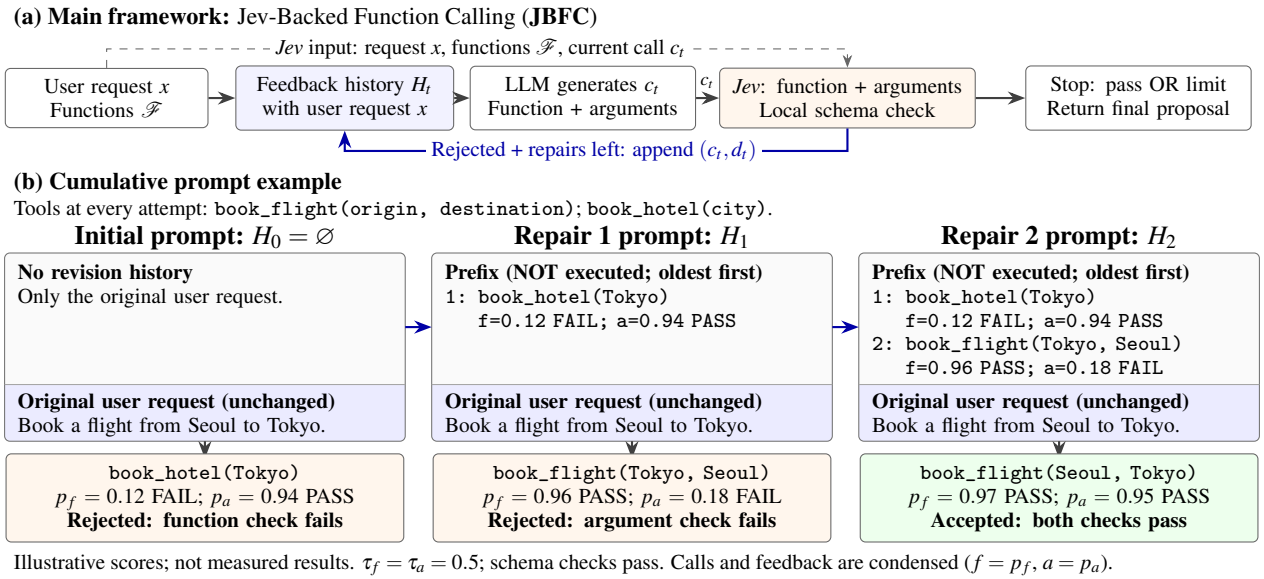


Figure 1: **JBFC** and cumulative prompt feedback. Top: the LLM reads earlier calls and judgments (H_t), then the original request (x), and generates a call (c_t) with a function and arguments. Failed checks add a record for another attempt. Bottom: growing history precedes the unchanged request; new calls appear below. Argument checks concern the proposed function. No function runs during repair.

Our records are likewise model-generated examples, labeled by *Jev* instead of reference answers.

***Jev* as a judge.** Li et al.'s *JEV-as-a-Judge* [15] checks finished outputs with probabilities for defined verdicts. If a verdict is uncertain, an independent, stronger judge decides. With fixed rules for when to use the stronger judge, it lowers judging cost while keeping similar accuracy on some tasks. It has limits on difficult reasoning, misleading answer styles, and writing judged without reference information. The study does not test tool or action selection inside agents. **JBFC** checks proposed function calls and returns failed checks to the same generator for revision. We test the specific design of **JBFC**.

3 Method

3.1 Propose and Check a Call

Let x be the user request, $\mathscr{F}$ the available functions, and G the generating LLM. A proposal is $c_t = (f_t, a_t)$, with function f_t and arguments a_t, or $c_t = \varnothing$ for no call. We study single-call-or-no-call decisions.[1] Local code checks readability, function availability, required arguments, and types. Write $S(c_t) = 1$ when these checks pass.

Binary verification: JBFC. For each readable proposal, one *Jev* request checks the function choice and, when a call is present, its arguments:

$$c_t = G(H_t, x, \mathscr{F}), \tag{1}$$

$$p_f^{(t)} = J_f(x, \mathscr{F}, c_t), \qquad p_a^{(t)} = J_a(x, \mathscr{F}, c_t). \tag{2}$$

Here H_t contains earlier proposals and feedback and is initially empty. The function `Noul` question asks whether the function choice, including no call, fits the request. The argument question checks required values, their support in the request, and their parameter roles. For no call, it is skipped and $p_a^{(t)}$ is undefined. A selected function with missing arguments still receives both checks. Both must pass for a call.

3.2 Accept or Repair a Call

Acceptance requires the schema check and all required *Jev* checks to pass on the same proposal. The acceptance rule is

$$A_t = \begin{cases} S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f], & c_t = \varnothing, \\ S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f]\,\mathbf{1}[p_a^{(t)} \geq \tau_a], & \text{otherwise.} \end{cases} \tag{3}$$

`Noul` returns the probability of yes, with no separate confidence field [13, 14]. We use $\tau_f = \tau_a = 0.5$, chosen after runs at 0.8 and 0.7 (Section 4.1).

Feedback before the user request. H_t stores earlier records. The current record (c_t, d_t) pairs proposal c_t with feedback d_t (probabilities, pass/fail flags, thresholds, and local errors). No-call skips the argument check. After rejection, if repairs remain, $\|$ adds $[(c_t, d_t)]$, a list containing this one record, to the end of H_t:

$$H_{t+1} = H_t \,\|\, [(c_t, d_t)]. \tag{4}$$

[1] We treat single-turn decisions as the basic unit of function calling. Parallel calling combines multiple such decisions, while multi-turn interactions repeat them with updated context. We therefore evaluate the verification and repair loop on this basic unit and exclude parallel and multi-turn settings from this study.

Algorithm 1 JBFC: verification and prompt repair

Require: Request x, functions $\mathscr{F}$, generator G, *Jev* J, repair limit r, thresholds τ_f, τ_a

$H \leftarrow [\,]$
for $t = 0, \ldots, r$ **do**
  $c \leftarrow G(\text{Prefix}(H, x), \mathscr{F})$
  $(s, e, \text{readable}) \leftarrow \text{SchemaCheck}(c, \mathscr{F})$
  $p_f, p_a \leftarrow \text{null}, \text{null}$
  if readable **then**
    $Q \leftarrow \{\text{function}\}$
    if $c \neq \varnothing$ **then** $Q \leftarrow Q \cup \{\text{arguments}\}$
    $(p_f, p_a) \leftarrow J(x, \mathscr{F}, c, Q)$
  end if
  $A \leftarrow \text{Accept}(s, c, p_f, p_a, \tau_f, \tau_a)$ ▷ Eq. 3
  $d \leftarrow \text{Feedback}(e, p_f, p_a, \tau_f, \tau_a, c)$
  if $A = 1$ **or** $t = r$ **then break**
  $H \leftarrow H \,\|\, [(c, d)]$
end for
return c **if** $A = 1$ **else** c_0 ▷ c_0: first proposal

The LLM reads this history before the unchanged request and can change both function and arguments. Schemas remain available; system instructions stay separate.

Algorithm 1 rechecks up to $r+1$ proposals: the first plus repairs ($r = 3$ by default and in experiments). Its `Prefix` function places JSON records, oldest first, before unchanged original messages. Empty history adds no prefix. Records include round numbers and mark proposals as not executed. Here $s = S(c)$ and e lists local errors. Readable proposals receive *Jev* checks even if their schema fails; unreadable ones receive local feedback. *Jev* never receives history. Unasked p_a stays null. Null required probabilities cannot pass the gate; invalid API responses stop the run. Reference answers stay out of model requests. API errors stop the run after transport retries, separate from repairs.

3.3 Final Proposal

When a proposal passes all applicable checks, we return it. If the repair limit is reached first, we return the first proposal, so a repair replaces the generator's answer only once it passes (Table 3 compares returning the last). We score the final proposal against the benchmark answers without running the function.

4 Experiments

4.1 Data and Evaluation

We prepare 3,051 BFCL examples from seven categories at repository revision `6ea57973c7a6`[2]. Each run uses all examples once. Settings were chosen on these examples (no held-out split): $r = 3$ on the first run at threshold 0.8, returning the last proposal; then threshold 0.5 (of 0.7 and 0.5) and the first proposal, after **JBFC** lowered `gpt-5.4-mini`'s accuracy.

The original Python abstract syntax tree (AST) checker compares function names and arguments with allowed answers, not execution success; irrelevance categories require no call. Live relevance checks only call presence, so the main score, a micro-average, covers 3,035 examples. BFCL's "multiple" categories offer several functions but no parallel calls; we exclude parallel, multi-turn, web-search, memory, and execution categories.

[2]Full revision: `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The saved setup lists source URLs, hashes, and format changes.

		`gpt-5-nano`		`gpt-5.4-mini`	
Category	Examples	Direct	**JBFC**	Direct	**JBFC**
Simple Python	400	90.8	91.3	90.5	**91.8**
Multiple	200	91.0	91.5	93.3	**95.0**
Irrelevance	240	27.2	78.2	87.1	**87.9**
Live simple	258	74.3	77.8	77.1	**79.8**
Live multiple	1053	67.6	71.4	75.2	**77.4**
Live irrelevance	884	48.8	65.7	**80.7**	79.4
Live relevance[†]	16	89.6	**91.7**	79.2	83.3
Total	3051	64.1±0.3	74.8±0.3	81.1±0.2	**82.1**±0.1
USD/1k		0.056	0.210	0.703	1.137

Table 1: Accuracy (%) on all BFCL examples, mean of three runs, $r = 3$. Bold: highest per row. Total: mean ± s.d. over runs. USD/1k: cost per 1,000 requests. [†]Call presence only; not in the total.

4.2 Models and Methods

The generator is the Azure OpenAI deployment `gpt-5-nano` (`gpt-5-nano-2025-08-07`), with reasoning effort `none` and at most 4,096 output tokens; *Jev* is pinned to `jev-1.13.0`. We use Chat Completions through an APIM gateway and System One directly, and convert BFCL schemas to JSON Schema with API-safe function names. Generations cut off at the output limit or blocked by the provider's filter count as wrong. Direct and **JBFC** also run on the larger, costlier deployment `gpt-5.4-mini` (snapshot 2026-03-17) with the same settings. Table 1 averages three runs per generator, each with new initial proposals.

We compare seven methods, sharing one initial proposal per example. *Direct* uses the initial proposal; *Schema-only* repairs schema errors. *Self-refine* uses generic self-review in all r repairs, not the full Self-Refine method [6]. **JBFC-F** and **JBFC-A** obtain applicable `Noul` judgments but expose and require only the function or argument verdict, plus schema checks. **JBFC-A** accepts no call on schema checks alone. **JBFC-L** shares **JBFC**'s settings and initial proposal but shows only the latest rejected proposal and feedback. Neither model sees r and nothing runs before the end, so a larger limit only continues trajectories: the first run ran all methods with $r = 2$ and continued them to $r = 3$; the *Jev* methods ran again at threshold 0.5 (stated in their feedback) from the same initial proposals, **JBFC** to $r = 10$ for Figure 2. Like **JBFC**, methods that stop at a passing proposal return the first if none passes.

4.3 Measures, Cost, and Uncertainty

We report call, function, and argument accuracy (the last given the correct function), fixes and new errors, acceptance, and repairs. We compare repairs with Direct, **JBFC** with **JBFC-L**, and the two final-proposal rules on matching examples and initial proposals, bootstrapping example IDs 2,000 times (seed 1729) for 95% intervals after averaging each ex-

Method	Call	Func.	Args.	Sec.	USD/1k
Direct	63.8	74.8	81.2	1.30	0.056
Schema repair	65.8	76.4	81.8	1.36	0.059
Self-refine	70.0	80.3	82.2	5.47	0.265
JBFC-F	73.2	84.5	81.4	2.24	0.172
JBFC-A	72.5	82.6	**82.7**	2.21	0.169
JBFC-L	73.5	84.3	82.2	2.60	0.203
JBFC	**74.6**	**85.3**	82.1	2.61	0.211

Table 2: Methods on all BFCL examples (one run, $r = 3$, `gpt-5-nano`). Bold: highest accuracy per column. Call and function accuracy (%) exclude live relevance; argument accuracy (%) uses calls with the correct function.

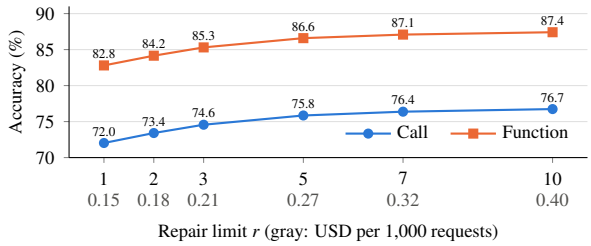


Figure 2: **JBFC** accuracy (%) by repair limit r: one run continued to $r = 10$ and cut at each limit ($r = 3$ is Table 2).

Generator	Direct	Last	First	First − last
`gpt-5-nano`	64.1	**75.1**	74.8	−0.35 [−0.68, −0.03]
`gpt-5.4-mini`	81.1	80.7	**82.1**	+1.37 [0.93, 1.82]

Table 3: Call accuracy (%), mean of three runs, when **JBFC** returns the last or the first proposal if none passes. First − last: paired difference [95% interval].

ample over runs.

Costs count all generation and verification work, including the growing history and the shared initial call; times are mean API seconds without retry waits. Prices in USD per million uncached/cached/output tokens are 0.05/0.005/0.40 for `gpt-5-nano` and 0.75/0.075/4.50 for `gpt-5.4-mini`; *Jev* input costs $0.042 (output free), without gateway fees; one filtered generation without usage is left out.

4.4 Results

In Table 1, **JBFC** raises mean accuracy from 64.1% to 74.8% (paired difference 10.7 points, 95% interval 9.7–11.6; per run 10.8, 10.7, 10.4). Per run, it fixes 323–337 wrong initial proposals, breaks 7–10, and averages 0.64–0.65 repairs; 84–85% of examples pass, 84–85% of them correctly. Other numbers are from the first run unless stated.

Returning the first proposal. The rules differ only when no proposal passes; returning the last then makes unverified changes, which per run fix 26–33 calls and break 16–22 with `gpt-5-nano` but fix 10–14 and break 53–55, mostly correct no-calls, with `gpt-5.4-mini`. Keeping only accepted changes gains 1.4 points for `gpt-5.4-mini` and costs `gpt-5-nano` 0.4 (Table 3).

A larger generator. Without *Jev*, `gpt-5.4-mini` reaches 81.1%, 6.4 points above **JBFC** with `gpt-5-nano` (95% interval 5.1–7.7), at 3.3 times the cost and under half the time. **JBFC** raises it in every run, by 1.0 point on average (95% interval 0.5–1.5); per run, it fixes 55–66 calls and breaks 27–39, mostly no-calls turned into accepted calls.

Comparison of methods. In Table 2, every repair method beats Direct (paired 95% intervals above zero): schema repair by 2.0 points, self-refine 6.3, **JBFC-F** 9.5, **JBFC-A** 8.7, and **JBFC-L** 9.8. Self-refine makes four proposals per request yet trails the *Jev*-based methods (1.4 to 1.6).

Why no-call cases gain most. Direct calls the one, unrelated function in 179 of 240 Irrelevance examples. *Jev* separates these from the 61 correct no-calls (median function probability 0.09 vs. 0.95), and the generator withholds 126 of 156 rejected calls; in live irrelevance, separation is weaker (0.53 vs. 0.87), and 160 of 379 are withheld. Where reference calls exist, **JBFC** rejects 216 of 463 wrong initial calls but fixes only 51: a verdict flags a wrong call, not the right one.

More repairs. In Figure 2, $r = 10$ reaches 76.7%, at 1.6 times the time and 1.9 times the cost of $r = 3$; later passing proposals fix 70 examples (59 by withholding a call) and break 4.

Records as in-context examples. Each record pairs a call with *Jev*'s verdict, so the history acts as labeled in-context examples, made without reference answers in 0.19 s per label at about $0.06 per 1,000. As in many-shot learning [17], more examples help: with the same attempts, the full history beats the latest record by 1.1 points (**JBFC** minus **JBFC-L**, 95% interval 0.4–1.7), though all are failed attempts at one request.

4.5 Study Limits

The verifier can accept a wrong call or share the generator's mistake, and feedback can introduce or reinforce errors. We use two generators, one verifier, and mostly English single-turn requests; Table 2 and Figure 2 come from one run, and public examples may appear in training data. Ten documented annotation errors stay. Settings were chosen on these examples, so gains may be optimistic; effort `none` is untuned, and one common limit may favor some methods.

5 Conclusion

Feeding back *Jev*'s judgments as a history, **JBFC** with three repairs raised mean accuracy from 64.1% to 74.8% with `gpt-5-nano`, mostly by withholding unnecessary calls, and from 81.1% to 82.1% with `gpt-5.4-mini`, as it returns the first proposal when none passes.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.

References

[1] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning*, PMLR 267, pp. 48371–48392, 2025. https://proceedings.mlr.press/v267/patil25a.html.

[2] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024. https://arxiv.org/abs/2305.15334.

[3] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.16789.

[4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. https://aclanthology.org/2023.emnlp-main.187/.

[5] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of ACL*, pp. 11143–11156, 2024. https://aclanthology.org/2024.findings-acl.664/.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. https://arxiv.org/abs/2303.17651.

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023. https://arxiv.org/abs/2303.11366.

[8] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2305.11738.

[9] Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. ToolVerifier: Generalization to New Tools via Self-Verification. *arXiv:2402.14158*, 2024. https://arxiv.org/abs/2402.14158.

[10] Hassan Hamad, Yingru Xu, Liang Zhao, Wenbo Yan, and Narendra Gyanchandani. ToolCritic: Detecting and Correcting Tool-Use Errors in Dialogue Systems. *arXiv:2510.17052*, 2025. https://arxiv.org/abs/2510.17052.

[11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.01798.

[12] TypeSafe. Models: *Jev* and the System One API. Technical documentation, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/models.

[13] TypeSafe. Noul. Technical documentation, 2026. Accessed October 17, 2026. https://docs.typesafe.ai/primitives/noul.

[14] TypeSafe. Confidence. Technical documentation, 2026. Accessed October 17, 2026. https://docs.typesafe.ai/confidence.

[15] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv:2609.26550*, version 5, October 9, 2026. https://arxiv.org/abs/2609.26550v5.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[17] Rishabh Agarwal, Avi Singh, Lei Zhang, Bernd Bohnet, Luis Rosias, Stephanie Chan, Biao Zhang, Ankesh Anand, Zaheer Abbas, Azade Nova, John D. Co-Reyes, Eric Chu, Feryal Behbahani, Aleksandra Faust, and Hugo Larochelle. Many-Shot In-Context Learning. In *Advances in Neural Information Processing Systems*, 37:76930–76966, 2024. https://arxiv.org/abs/2404.11018.